Exercise M2 – Implementing and Evaluating NLP Models

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract* — This study explores applying Natural Language Processing (NLP) techniques to the realm of sports journalism with the production of four models, text classification, sentiments analysis, part-of-speech tagging, and text summarization. Pre-processed sports articles was used and manually annotated data sets of sports articles to both train, test, and evaluate each model and to benchmark each against an average accuracy of 80%. The performance results indicate that all of our models performed at or above the average accuracy of 80% with text classification accuracy reaching 92%, sentiment analysis at 88%, part-of-speech tagging at 84.6%, and text summarization at an average ROUGE-1 F1 score of 88.8%. As demonstrated, the significance of data pre-processing, data set annotation, and testing and evaluation cannot be undervalued in achieving valid and trustworthy results. The study shows potential of employing natural language processing pipelines that can be effective in certain domains. The study also provided a foundation for future developments to enhance future practice.**

Keywords—part of speech tagging, sports journalism, text classification, sentiment analysis, text summarization

I. INTRODUCTION

Natural Language Processing (NLP) has fundamentally altered the course of computational linguistics by allowing machines to accept, interpret, and generate human languages. The exercises in this paper utilize various methods of NLP implemented in the programming language Python, specifically on sports-related text data. The goal of this exercise is to provide the practical application and discussion about NLP concepts such as text classification, sentiment analysis, part-of-speech tagging, and summarization, while emphasizing good documentation and technical communication.

The datasets utilized in this paper consisted of sports articles found online and a manually labeled dataset of emails related to sports. The manually labeled dataset included 60 entries that were created to train models on domain-specific tasks and had been ensured to be applicable and contextually correct for the purposes of the study. Together, these datasets were used to gain an understanding of the results of the NLP models on both structured and unstructured sports text, and focused on collection data and model performance metrics such as Accuracy, Precision, Recall, and F1-Score. This work not only showcased the practical application of NLP pipelines but also emphasized the need for model evaluation, good preprocessing documentation and technical documentation for reproducible and correct results and reproducibility of modern research for everyone else.

In addition to technical implementations, NLP plays an important role in improving the integrity and sport texts. By analyzing text data produced by different data sources such as news articles, social media, and internal communications, NLP provides ways to help analyze anomalies, trust and safety, and any potential threats to integrity such as match fixing or doping scandals. These uses will help preserve fair play and an open and broad transparency in sport. NLP uses can also provide unique insights dealing with improved fan engagement through personalized content and real-time updates, as well as automating mundane processes like summarizing match reports or classifying sports news. Thus, NLP has a role to help advance not just analytic capabilities but also ethical and operational form of the sport [1].

II. METHODOLOGY

In this study, sports articles and a human labeled dataset of sport-related emails are utilized with preprocessing steps, including cleaning, tokenization, lemmatization and normalization to get it ready for analysis. The key NLP tasks include text classification to categorize the sports-related spam, part of speech tagging to analyze each article's grammatical structure, sentiment analysis to get the text's tone or sentiment, and text summarization to achieve a summar. These conclusions provide methods to process and process text-based specific sport-related data.

a) Sourced Dataset Preprocessing

The sports articles that were gathered went through an extensive pre-processing pipeline in order to convert the raw text into a cleaned and standardized text data for NLP tasks. It was represented in all lowercase text for consistency and lowered vocabulary sparsity. Also, contractions for example "can't" were also decompressed to "cannot" with the intent of semantic transparency. The removal of special characters, URLs, and additional punctuation ensured that noise that could confound tokenization, or learning by the model, was no longer there.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Stop words were also removed so the models would not be distracted with inconsequential words.

Text was then tokenized, which is to say, sentences were broken down into individual words so as to allow for exact feature extraction, on top of this, lemmatization was applied to reduce inflected words to their root words, for example "running" to "run", ensuring that different parts of the same word are treated as the same unit of measurement.

Together, all these efforts produced a cleaned and standardized dataset which will be useful for any number of downstream tasks, including text classification, sentiment analysis, and summarization. Additionally, a column for POS tagging was created based on the tokenized text, and a summary column was created to facilitate evaluation of text summarization in future tasks. The dataset that was utilized was the sports_data .csv from the News Articles Classification Dataset for NLP & ML located on Kaggle [2].

raw_content	content	tokenized	lemmatized	gold_pos	summary
In a resolute, nerveless,	resolute, nerveless, three-hour-l	['resolute', 'nerveles	['resolute', 'nerveless',	['VERB', 'ADJ', 'NOUN'	Scintillatin
Legendary Australia	legendary australia cricketer allar	['legendary', 'austral	['legendary', 'australia',	['PROPN', 'PROPN', 'P	and we
Jannik Sinner has stunned	jannik sinner stunned novak djok	['jannik', 'sinner', 'stu	['jannik', 'sinner', 'stun',	['PROPN', 'PROPN', 'V	Janik
England opener Ben	england opener ben duckett says	['england', 'opener',	['england', 'opener', 'be	['PROPN', 'PROPN', 'P	Hopefully,
Since the start of 2023,	since start 2023, kiran george pla	['since', 'start', 'kiran	['since', 'start', 'kiran', 'g	['SCONJ', 'PROPN', 'PR	Winning
Naushad Khan was	naushad khan flooded congratula	['naushad', 'khan', 'fl	['naushad', 'khan', 'flooc	['VERB', 'PROPN', 'VER	Sarfaraz,
For Novak Djokovic,	novak djokovic, hunting record-e	['novak', 'djokovic', 'h	['novak', 'djokovic', 'hun	['PROPN', 'PROPN', 'V	One game
At the end of Day One, of	end day one, series promises ple	['end', 'day', 'one', 'se	['end', 'day', 'one', 'serie	['NOUN', 'NOUN', 'NU	In U-19
Bangladesh allrounder	bangladesh allrounder shakib al h	['bangladesh', 'allrou	['bangladesh', 'allround	['NOUN', 'NOUN', 'PR	The
For a year that saw him	year saw finish highest run score	['year', 'saw', 'finish',	['year', 'saw', 'finish', 'hi	['NOUN', 'AUX', 'VERB	In U-19
More than the roar from	roar 24,000 odd spectators jubilar	['roar', 'odd', 'spectat	['roar', 'odd', 'spectator',	['VERB', 'ADJ', 'NOUN'	Batted,
Australian allrounder	australian allrounder cameron gr	['australian', 'allroun	['australian', 'allrounder	['ADJ', 'NOUN', 'PROP	Starc made
Monty Panesarâ€™s	monty panesars wicket sachin te	['monty', 'panesars',	['monty', 'panesars', 'wi	['PROPN', 'NOUN', 'NO	When they
Rohan Bopanna entered	rohan bopanna entered final aus	['rohan', 'bopanna', 'e	['rohan', 'bopanna', 'ent	['PROPN', 'PROPN', 'V	Bopanna
Indian Olympic and world	indian olympic world champion r	['indian', 'olympic', 'w	['indian', 'olympic', 'wor	['ADJ', 'ADJ', 'PROPN',	Indian
James Anderson would	james anderson would begin sixt	['james', 'anderson',	['james', 'anderson', 'wo	['PROPN', 'PROPN', 'A	James And
The arrival of Bazball and	arrival bazball fixation around me	['arrival', 'bazball', 'fi	['arrival', 'bazball', 'fixat	['NOUN', 'NOUN', 'NO	So much
Dinesh Karthik was	dinesh karthik among first ones e	['dinesh', 'karthik', 'a	['dinesh', 'karthik', 'amo	['PROPN', 'PROPN', 'A	Karthik
A WTT Star Contender	wtt star contender held india sta	['wtt', 'star', 'contenc	['wtt', 'star', 'contender'	['PROPN', 'PROPN', 'P	Final

Fig. 1. Preprocessed sourced dataset sample rows

b) Manual dataset processing

The sports emails labeled manually also underwent a method of preprocessing similar to what was done for our sourced sports articles. Specifically, all of the text were converted to lower case, strip out all of the contractions, strip out special characters and punctuation, stopped words filtering, tokenization, lemmatization, which reduces the word to its base form. This preprocessing step ensures the text is clean, consistent, and standardized, which is needed in a manually created dataset, even.

Data cleaning and preprocessing matter, because input data quality has a direct effect on machine learning models' performance and reliability. Even with manually labeled data, inconsistencies, typos, or other noise exist, these discrepancies may lead to inaccurate or biased predictions. By preprocessing the data, we reduce dimensional complexity, cleanse noise from the data and prepare raw text for the way it needs to be presented in order for any NLP algorithms to learn effectively. Data preprocessing enhances model accuracy, improves reproducibility, and increases results interpretability, making it fundamental step in any NLP pipeline [3].

raw_text	text	tokenized	lemmatized	spam_label	sentiment_label
0||Congratulations! Youâ€™ve been sel	"congratulations! selected exclusive chance win 10 free tickets nb	['congratulations', 'selected	['congratulation', 'select',	SPAM	POSITIVE
0||Claim your VIP pass to the World Cup	"claim vip pass world cup today-limited slots only! act fast, lose ch	['claim', 'vip', 'pass', 'world',	['claim', 'vip', 'pas', 'world	SPAM	NEGATIVE
0||You just won an all-expense-paid trip	"you all-expense-paid trip see champions league final. confirm de	['trip', 'see', 'champions', 'le	['trip', 'see', 'champion', 'l	SPAM	POSITIVE
0||This is your last chance to register for	"this last chance register jackpot raffle sports merchandise worth	['last', 'chance', 'register', 'ja	['last', 'chance', 'register',	SPAM	NEGATIVE
0||Great news! Your email has been dra	"great news! email drawn receive free season passes team lakers	['great', 'news', 'email', 'dra	['great', 'news', 'email', 'd	SPAM	POSITIVE
0||Exclusive betting bonus: deposit now	"exclusive betting bonus deposit triple money tonight's match. le	['exclusive', 'betting', 'bonu	['exclusive', 'bet', 'bonus',	SPAM	NEGATIVE
0||You have unlocked premium access t	"you unlocked premium access behind-the-scenes footage favorit	['unlocked', 'premium', 'acc	['unlocked', 'premium', 'a	SPAM	POSITIVE
0||Act now to claim your exclusive F1 pad	"act claim exclusive f1 paddock tickets - hidden fees drain account	['act', 'claim', 'exclusive', 'pa	['act', 'claim', 'exclusive',	SPAM	NEGATIVE
0||Warning: your prize claim will be voi	"warning prize claim voided fail verify within 24 hours. final notic	['warning', 'prize', 'claim', 'w	['warn', 'prize', 'claim', 'vo	SPAM	NEGATIVE
0||Hooray! You are guaranteed a spot at	"hooray! guaranteed spot super bowl halftime show. complete ra	['hooray', 'guaranteed', 'spo	['hooray', 'guaranteed', 'spo	SPAM	POSITIVE
0||Special alert: thousands are already s	"special alert thousands already cashing free betting credits. one	['special', 'alert', 'thousands	['special', 'alert', 'thousan	SPAM	NEGATIVE
0||You'll â€™ve qualified for the Golden F	"you qualified golden ticket giveaway-win exclusive courtside se	['qualified', 'golden', 'ticket'	['qualify', 'golden', 'ticket	SPAM	POSITIVE
0||Final reminder: your free VIP upgrad	"final reminder free vip upgrade tonight's big fight canceled confi	['final', 'reminder', 'free', 'vi	['final', 'reminder', 'free',	SPAM	NEGATIVE
0||Congratulations! Youâ€™ve been cho	"congratulations! chosen free sports streaming package, including	['congratulations', 'chosen',	['congratulation', 'choose	SPAM	POSITIVE
0||Urgent: This offer will expire tonight	"urgent offer expire tonight. redeem chance win 10,000 betting cr	['urgent', 'offer', 'expire', 'to	['urgent', 'offer', 'expire',	SPAM	NEGATIVE
0||The Golden State Warriors secured a	"the golden state warriors secured thrilling overtime victory last r	['golden', 'state', 'warriors',	['golden', 'state', 'warrior'	HAM	POSITIVE
0||Manchester United's poor form co	"manchester united's poor form continued suffered disappointing	['manchester', 'united', 'poo	['manchester', 'united', 'p	HAM	NEGATIVE
0||The city was left frustrated as the F1 Gra	"the city left frustrated f1 grand prix faced another cancellation di	['city', 'left', 'frustrated', 'gra	['city', 'leave', 'frustrated'	HAM	NEGATIVE
0||Serena Williams announced her with	"serena williams announced withdrawal tournament sustaining h	['serena', 'williams', 'annou	['serena', 'williams', 'ann	HAM	NEGATIVE
0||The Los Angeles Dodgers dominated	"the los angeles dodgers dominated field, hitting four home runs.	['los', 'angeles', 'dodgers', 'd	['los', 'angeles', 'dodger',	HAM	POSITIVE
0||Despite high expectations, the natio	"despite high expectations, national football squad crashed qualif	['despite', 'high', 'expectatio	['despite', 'high', 'expecta	HAM	NEGATIVE
0||Fans celebrated as the rookie striker	"fans celebrated rookie striker scored first professional goal, secu	['fans', 'celebrated', 'rookie'	['fan', 'celebrate', 'rookie'	HAM	POSITIVE
0||A controversial referee decision ove	"a controversial referee decision overshadowed exciting clash rea	['controversial', 'referee', 'd	['controversial', 'referee',	HAM	NEGATIVE

Fig. 2. Preprocessed manual dataset sample rows

Both spam classification and sentiment analysis labels were intentionally balanced to keep bias conditionally based on our NLP tasks. By having both classes approximately balanced, the models had no bias to a majority class and we were able to produce more reliable performance measures, and the models can generalize well to the unseen data.

c) Text Classification Model

Text classification is a key task of a Natural Language Processing (NLP) research area which involves assigning text data to predetermined categories or labels. The importance of understanding and studying text classification cannot be overstated because it is a baseline used for many real-world applications, for example, sentiment analysis, spam detection, topic classification, or content moderation. The technology around text classification has changed so dramatically with state-of-the-art language models, representation models such as BERT, and generative models like ChatGPT, which are capable of learning complex patterns, contexts, and semantics of language at an internal level, up to and including classification tasks. This allows for more accurate and efficient text classification with even a small set of labeled data, as well as zero-shot learning of categories where no training has been conducted on similar labeled categories. Developing text classification skills enables practitioners to automate processing of large sets of text while also extracting meaningful insights, which improves decision-making across a variety of settings [4].

In this study, the researcher introduced a simple binary text classification model that utilizes emails as classified documents for SPAM and HAM categories. The model used the TextCategorizer pipeline built in spaCy that follows neural network architecture for natural language processing. The TextCategorizer converts each email into vector representations and learns the relationships between textual patterns and the class labels through supervised learning of email documents.

The implementation in this is similar but has a few notable improvements compared to the original sample code example. First, the code did not utilize hardcoded examples, but loaded a custom dataset (ExerciseM2_YORO_spam_sentiment_cleaned.csv) that was given manually labeled SPAM and HAM labels. Each row of the dataset was able to be converted into Example

objects compatible with spaCy that allowed the model to be trained on realistic, domain-specific email documentation.

To encourage the models' generalization, the sample and original code performed model evaluation on training examples that were also used to train; and by evaluating performance on the training examples, it significantly increased risk of overfitting and inflated performance metrics, especially since SPAM emails are relatively similar. The dataset was split into training (60%) and testing (40%) dataset subsets using train_test_split.

```python
import spacy
from spacy.training import Example
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

# --------------------------
# Load blank English model
# --------------------------
nlpTC = spacy.blank("en")

# Add TextCategorizer to the pipeline
textcat = nlpTC.add_pipe("textcat")

# Add labels
textcat.add_label("SPAM")
textcat.add_label("HAM")

# --------------------------
# Load dataset from CSV
# --------------------------
csv_file = "ExerciseM2_YORO_spam_sentiment_cleaned.csv"
df = pd.read_csv(csv_file)


df['spam_label'] = df['spam_label'].str.upper()

# --------------------------
# Split dataset into train/test
# --------------------------
train_df, test_df = train_test_split(df, test_size=0.4, random_state=42)
```

Fig. 3. Dataset initialization to text classification model

Hyperparameter tuning further refined the pipeline:

- Number of iterations (n_iter = 10) -The number of passes the model will make over the training data.
- Dropout (drop = 0.3)- Randomly turn off some fraction of neurons on each update to reduce overfitting.
- Test size (test_size = 0.4) - Ensure you have set aside enough data to evaluate the model performance.

The model adjusted weights based on the examples in the training set with dropout applied. This is not completed in the original example code. Then, the model performance was evaluated and measured on the test dataset using Accuracy, Precision, Recall, and F1-Score. The confusion matrix created could be viewed using matplotlib and seaborn to see true positive, true negative, false positive, and false negative: a valuable measure which is not displayed in the original code.

```python
train_df, test_df = train_test_split(df, test_size=0.4, random_state=42)

# Prepare training data
train_data = []
for idx, row in train_df.iterrows():
    label = row['spam_label']
    cats = {"SPAM": 0, "HAM": 0}
    cats[label] = 1
    train_data.append((row['text'], {"cats": cats}))

# Prepare testing data
test_data = []
for idx, row in test_df.iterrows():
    label = row['spam_label']
    cats = {"SPAM": 0, "HAM": 0}
    cats[label] = 1
    test_data.append((row['text'], {"cats": cats}))

# --------------------------
# Initialize optimizer and hyperparameters
# --------------------------
optimizer = nlpTC.initialize()
n_iter = 10

# --------------------------
# Training loop
# --------------------------
for i in range(n_iter):
    for text, annotations in train_data:
        example = Example.from_dict(nlpTC.make_doc(text), annotations)
        nlpTC.update([example], sgd=optimizer, drop = 0.3)

# --------------------------
# Evaluation on test data
# --------------------------
y_true = []
y_pred = []

for text, annotations in test_data:
    doc = nlpTC(text)
    predicted_label = "SPAM" if doc.cats["SPAM"] > doc.cats["HAM"] else "HAM"
    y_pred.append(predicted_label)
    true_label = "SPAM" if annotations['cats']["SPAM"] == 1 else "HAM"
    y_true.append(true_label)
```

Fig. 4. Train/test split to test data evaluation of text classification model

```python
# --------------------------
# Compute Evaluation Metrics
# --------------------------
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label="SPAM")
recall = recall_score(y_true, y_pred, pos_label="SPAM")
f1 = f1_score(y_true, y_pred, pos_label="SPAM")

print("Evaluation Metrics on Test Data:")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")

# --------------------------
# Confusion Matrix Visualization
# --------------------------
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix


cm = confusion_matrix(y_true, y_pred, labels=["SPAM", "HAM"])


plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=["SPAM", "HAM"], yticklabels=["SPAM", "HAM"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```

Fig. 5. Computation of evaluation metrics and visualization

These changes are warranted because they take the original prototype and shape it with reliability and reproducibility as an NLP pipeline:

- The training sets with a realistic dataset enhances domain relevance.
- Train/test splits allow monitoring for overfitting and generally signify an accurate way of modelling to measure performance.
- Hyperparameters, such as dropout and iterations, give a degree of control over training learning as well as parameters to prevent generalization.
- Evaluation metrics and then the visualizations offer qualitative metrics that a more quantitative manner often leads to interpretability and academic rigor checks.

These changes allow the model to be practical and it may be considered reliable in reporting, producing results that are interpretable and measurable in research or applied environment.

d) Sentiment Analysis Model

Sentiment analysis is an important NLP technique for understanding opinions and emotions communicated in sports text (such as comments from fans, reports on games, and posts shared on social media). With its ability to classify sentiments in the text as either positive, negative or neutral, sentiment analysis can provide important insights into the public response to teams, players, and events. There are a range of methods, from the relatively simple rule-based system, to more complex machine learning and deep learning models, often augmented with brand- and domain-specific sentiment lexicons. Sentiment analysis is valuable for sports organizations to track fan engagement (positive or negative), identify emerging issues, and even modernize their communication strategies. It will provide organizations with the data regarding how their brand is being perceived, Fan sentiment analysis can promote and support data-based decisions to develop strategies to improve fan experience, reputation management, brand loyalty and fan engagement within a sports context, illustrating the value of sentiment analysis in extracting actionable insights from text data in sport [5].

The sentiment analysis model was performed using spaCy's TextCategorizer which is a supervised machine learning pipeline component for classification problems. The goal was to classify the sports texts, as POSITIVE or NEGATIVE. Each training example consists of a raw text input with a label as a dictionary, e.g., {"POSITIVE": 1, "NEGATIVE": 0}. TextCategorizer uses this label (returning a probability distribution over the two classes) to predict the sentiment.

The dataset used in this study contained 60 sports texts, equivalent in number across the POSITIVE and NEGATIVE classes. The raw text was prepared in the raw_text column and sentiment in the sentiment_label column. One key design decision was to work with the raw text as opposed to highly preprocessed text (namely, lowercased, lemmatized, removed stop words), since initial exploratory analysis indicated excessive preprocessing removed sentiment words which could have decreased classification accuracy, especially with a small dataset.

```
import spacy
from spacy.training import Example
import pandas as pd
import random
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_ma
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

# ---------------------------
# Load blank English model
# ---------------------------
nlp = spacy.blank("en")

textcat = nlp.add_pipe("textcat")

textcat.add_label("POSITIVE")
textcat.add_label("NEGATIVE")

# ---------------------------
# Load dataset from CSV
# ---------------------------
csv_file = "ExerciseM2_YORO_spam_sentiment_cleaned.csv"
df = pd.read_csv(csv_file)

df['sentiment_label'] = df['sentiment_label'].str.upper()

# ---------------------------
# Split dataset into train/test
# ---------------------------
train_df, test_df = train_test_split(df, test_size=0.4, random_state=42)
```

Fig. 6. Dataset initialization to train/test split of sentiment analysis model

The dataset was divided into training and testing subsets at a 60:40 ratio. Each text-label pair was converted into a spaCy Example object in order to be properly processed by the TextCategorizer. Hyperparameters had been carefully fine-tuned for what is already a small dataset: 60 training iterations were ran so as to allow enough learning, and a dropout rate of 0.2 so as to not under-regularize the model, both of which seemed to work for the limited examples available for learning. The training loop updated the model weights with stochastic gradient descent on the shuffled examples.

The model was assessed on the test set with accuracy, precision, recall, and F1-score metrics, providing a solid assessment of performance. A confusion matrix was also generated and visualized with seaborn for easy review of classifications that were misclassified. For classifying the user input of new texts, a real-time loop was implemented that stripped the outputs down to POSITIVE or NEGATIVE labels, rather than full probability distributions, to improve interpretability.

```
train_data = []
for idx, row in train_df.iterrows():
    label = row['sentiment_label']
    cats = {"POSITIVE": 0, "NEGATIVE": 0}
    cats[label] = 1
    train_data.append((row['raw_text'], {"cats": cats}))

test_data = []
for idx, row in test_df.iterrows():
    label = row['sentiment_label']
    cats = {"POSITIVE": 0, "NEGATIVE": 0}
    cats[label] = 1
    test_data.append((row['raw_text'], {"cats": cats}))

# ---------------------------
# Initialize optimizer and hyperparameters
# ---------------------------
optimizer = nlp.initialize()
n_iter = 50
dropout_rate = 0.2

# ---------------------------
# Training loop
# ---------------------------
for i in range(n_iter):
    random.shuffle(train_data)
    for text, annotations in train_data:
        example = Example.from_dict(nlp.make_doc(text), annotations)
        nlp.update([example], sgd=optimizer, drop=dropout_rate)
```

Fig. 7. Sentiment analysis model hyperparameters and training loop

```python
# ---------------------------
# Evaluation on test data
# ---------------------------
y_true = []
y_pred = []

for text, annotations in test_data:
    doc = nlp(text)
    predicted_label = "POSITIVE" if doc.cats["POSITIVE"] > doc.cats["NEGATIVE"] else "NEGATIVE
    y_pred.append(predicted_label)
    true_label = "POSITIVE" if annotations['cats']["POSITIVE"] == 1 else "NEGATIVE"
    y_true.append(true_label)

# ---------------------------
# Compute Evaluation Metrics
# ---------------------------
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label="POSITIVE")
recall = recall_score(y_true, y_pred, pos_label="POSITIVE")
f1 = f1_score(y_true, y_pred, pos_label="POSITIVE")

print("Evaluation Metrics on Test Data:")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")

# ---------------------------
# Confusion Matrix Visualization
# ---------------------------
cm = confusion_matrix(y_true, y_pred, labels=["POSITIVE", "NEGATIVE"])
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=["POSITIVE", "NEGATIVE"], ytick
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show(block=False)
plt.pause(0.1)
plt.close()
```

Fig. 8. Test data evaluation and Evaluation matrixes and Visualization

There were several changes to the reference code provided:

- Replaced preprocessed text with raw text to maintain sentiment cues contextually.
- Hyperparameter tuning, by increasing iterations and reducing dropout, to be appropriate for the dataset size.
- Evaluation criteria and confusion matrix visualization, which would allow for a quantitative assessment of performance on model evaluation.
- Simplified prediction output, to only reveal the final sentiment label as it would be more user-friendly.

All modifications were justified as these addressed fundamental issues of small dataset learning, and enhanced sentiment signals and context, to help both the accuracy and explanation of the model. The final implementation was able to produce high performance metrics, while demonstrating the model is capable of accurately classifying sports texts and provided real-time classification of incoming inputs.

e) Part of Speech Tagging Model

Part-of-speech (POS) tagging is critical when analyzing sports texts, as it involves labeling and annotating words with their grammatical function. The objective of this function allows AI models to utilize POS tags to better understand the structure and meaning of the sentence. POS tagging can help clarify word meanings when seeking to analyze player actions, game events, and fan comments towards events that enables more effective task completion related to the sports domain such as sentiment analysis and event extraction of text. POS tagging lowers ambiguity of meaning as well as capturing and interpreting language at a deeper level. All of which improves accuracy beyond just determinants of POS tagging. Additionally, by using POS tagging, models will be more efficient when processing sport-based language for analytic or communicative purposes [6].

The POS tagging model used in this exercise is based on spaCy's pre-trained en_core_web_sm pipeline for English text, which implements tokenization and statistical POS tagging. The original sample code allowed user input to obtained POS tags at the token level, but to evaluate the model properly, we needed an evaluation process.

```python
import spacy
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ------------------- Original code (unchanged) -------------------
nlp = spacy.load("en_core_web_sm")

def analyze_text(text):
    doc = nlp(text)
    pos_list = [(token.text, token.pos_) for token in doc]
    return pos_list


user_input = input("Enter a text for POS tagging analysis: ")
pos_tags = analyze_text(user_input)
print("\nTokens and POS Tags:")
print(pos_tags)
```

Fig. 9. Retained sample code for PoS tagging model

A sample of 20 rows was manually selected, totaling about 1,000 tokens, and after determining each token in the subsample we assigned gold POS tags to provide a reference point. For each of the 20 rows, the text was tokenized into a processed input and passed to analyze_text() for getting predicted POS tags, which were then flattened along with the gold tags for token-level comparison.

To quantify performance across multiple perspectives, the evaluation metrics of Accuracy, Precision, Recall, and F1-Score were calculated. Accuracy indicates the percentage of correctly tagged tokens. Precision shows, of all predicted tags, how many were correct. Recall indicates how many true tags were accurately detected. F1-Score is a harmonic mean of precision and recall.

```python
# ------------------- Evaluation part -------------------

df = pd.read_csv("ExerciseM2_YORO.csv")


eval_df = df.dropna(subset=['gold_pos']).copy()

all_true = []
all_pred = []

for idx, row in eval_df.iterrows():

    tokens = eval(row['tokenized']) if isinstance(row['tokenized'], str) else row['tokenized']
    text_input = " ".join(tokens)

    pos_list = analyze_text(text_input)
    predicted_tags = [str(tag) for _, tag in pos_list]

    # Get gold tags as list of strings
    gold_tags = eval(row['gold_pos']) if isinstance(row['gold_pos'], str) else row['gold_pos']
    gold_tags = [str(tag) for tag in gold_tags]

    all_true.extend(gold_tags)
    all_pred.extend(predicted_tags)

# Compute metrics
accuracy = accuracy_score(all_true, all_pred)
precision = precision_score(all_true, all_pred, average='macro', zero_division=0)
recall = recall_score(all_true, all_pred, average='macro', zero_division=0)
f1 = f1_score(all_true, all_pred, average='macro')

print("Token-level POS Tagging Evaluation Metrics (first 20 rows):")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

Fig. 10. PoS tagging evaluation section

These changes maintained the original functionality of the code, while facilitating a consistent and reproducible way of evaluating POS tagging performance on a representative, human-reviewed subset that illustrate the efficacy of the pre-trained model at the token level and provided a reasonable quantitative assessment without requiring comprehensive labelling of the full dataset.

f) Text Summarization Model

NLP text summarization is a great way to reduce long texts in sport, including match reports, player interviews, and commentary from fans, into a concise summary that expresses important information and context. Summarization in text can be done through extractive and abstractive methods where extractive approaches select important sentences verbatim from the original text and abstractive approaches paraphrase the original text and reconstruct those sentences in a new way that conveys the same overall meaning, similar to how a human would summarize. This capability has important implications for sport, because greater efficiencies in the ability to extract valuable insight from large amounts of text may create opportunities to enhance analytical, coaching and fan-based decision-making. Summarizers that automate the summarization process can also save time, provide assurance that important details will not be neglected, and allow stakeholders to stay informed of game situations, player performance and sector news without having to read through lengthy texts. [7]

The model for text summarization built in this study is the extractive summarization approach; based on token frequency and sentence scoring; where the spaCy NLP library was used. The en_core_web_sm model was used to tokenize the input abstract into sentences and words and to filter-out stop words and punctuation. For each sentence contained in the document, a score is applied based on the total token counts of non-stop-word tokens associated with each sentence. The top N sentences selected based on individual sentence scores will be used to make up the summary. This approach is simple and more interpretable, and can easily fit into an NLP pipeline in Python, which makes it a practical model for educational purposes and experimentation.

```python
import spacy
from collections import Counter
import pandas as pd
from rouge_score import rouge_scorer

# ------------------- Original code (unchanged) -------------------

# -------------------------------
# Load spaCy model
# -------------------------------
nlp = spacy.load("en_core_web_sm")

# -------------------------------
# Summarization function
# -------------------------------
def summarize(text, n_sentences=2):
    doc = nlp(text)
    sentence_scores = Counter()

    for sent in doc.sents:
        for token in sent:
            if not token.is_stop and not token.is_punct:
                sentence_scores[sent] += 1

    top_sentences = [sent.text for sent, score in sentence_scores.most_common(n_sentences)]
    return " ".join(top_sentences)

# -------------------------------
# User input summarization
# -------------------------------
user_text = input("Enter the text you want to summarize: ")
summary = summarize(user_text)
print("\nSummary:")
print(summary)
```

Fig. 11. Text Summarization Sample Code Implementation

The method closely followed the provided sample code that allowed a user to enter users to evaluate concepts for the abstract with real-time summarization. However, for summarization evaluation of the summaries to occur, changes were made:

- Dataset Integration: The pipeline was expanded to load a CSV dataset containing preprocessed text (raw_content) and curated by hand reference summaries (summaries) that users can use for comparison.

- Automated Summarization for Several Rows: Rather than summarizing just user input, the model produces summaries for a sample of the dataset (the first 20 rows for starters) so we can evaluate it.

- Evaluation Metric Addition: A ROUGE-1 F1 score was added to measure the quality of the generated summaries against the reference summaries. A F1 score was needed because, unlike classification tasks, text summarization doesn't have discrete labels to categorize data to measure, so metrics like Accuracy or F1-Score can't be used at the token or sentence-level.

- Preservation of Original Sample Code: The summarizer itself, the handling of user input, and the sentence scoring process all remained the same. All of the additions were made outside of the original sample to keep its objective as an educational sample and not overwrite the logic of the original object.

The main challenge faced in performing this adaptation was the evaluation of an extractive summarization model, where unlike in classification models, there is no discrete ground truth labels for summaries. Since summaries are subjective in nature, they may differ in phrasing, order, or selection of sentences. To help mitigate this issue, the established evaluation metric for summarization research, ROUGE, proposes a robust

evaluation metric by identifying any overlapping unigrams, bigrams, or bigger sequences between the summary generated by the model being evaluated and the reference summary. In this research, we selected ROUGE-1 to measure lexical overlaps, which was appropriate due to the fact that the summaries produced by the model were also extractive in nature. Taking the average ROUGE-1 score for each row provided a meaningful performance indicator that was analogous to, "accuracy" as it relates to being truthful to evaluation methods in the subject area of research.

```python
# ------------------------------
# Hyperparameter
# ------------------------------
n_sentences = 2

# ------------------------------
# Generate summaries for first 20 rows
# ------------------------------
generated_summaries = []
for text in df[text_column][:20]:
    generated_summaries.append(summarize(text, n_sentences=n_sentences))

df.loc[:19, 'generated_summary'] = generated_summaries

# ------------------------------
# ROUGE-1 evaluation
# ------------------------------
scorer = rouge_scorer.RougeScorer(['rouge1'], use_stemmer=True)
rouge1_results = []

for i in range(20):
    reference = df.loc[i, reference_column]
    generated = df.loc[i, 'generated_summary']
    score = scorer.score(reference, generated)['rouge1'].fmeasure
    rouge1_results.append(score)

df.loc[:19, 'rouge1_f1'] = rouge1_results


# ------------------------------
# Print ROUGE-1 results
# ------------------------------
print("\nROUGE-1 Evaluation Results (First 20 Rows):\n")
for i, score in enumerate(rouge1_results):
    print(f"Row {i+1}: ROUGE-1 F1 = {score:.3f}")

avg_rouge1 = df['rouge1_f1'][:20].mean()
print(f"Average ROUGE-1 F1 for first 20 summaries: {avg_rouge1:.3f}")
```

Fig. 12. Text Summarization ROUGE Evaluation Implementation

These modifications provide a compromise between keeping the original educational sample code intact and having a defensible quantitative evaluation framework for the summarization model. The example illustrates both the theoretical challenges of rigorously evaluating NLP summarization performance and a defensibly rigorous way of evaluating results without discrete labels provided.

III. Key Findings and Observations

a) Text Classification Model

The text classification model performed well on the test dataset, with an accuracy of 0.92, precision of 1.00, recall of 0.86, and an F1-Score of 0.92. This indicates that the model performs well in predicting SPAM vs HAM emails. The precision of 1.00 meant that all emails predicted to be SPAM occurred to be SPAM and thus, strangely, no legitimate email was wrongly classified as SPAM. The slightly lower recall indicates that a small proportion of spam emails were missed. This is likely because of changes in the spam content or possibly a small number of training examples. The F1-Score here balances precision with recall and is also consistent with how well the model performed overall.

The confusion matrix depicts a more nuanced view of the model's predictions. Overall, we correctly discovered 12 SPAM emails from the total test emails and 10 HAM emails. We wrongly classified just 2 HAM emails as SPAM and made no false positives - correctly classified all SPAM emails as SPAM. This distribution explains why we had high precision and slightly lower recall, as the model was conservative with its SPAM predictions in order to avoid false positives.

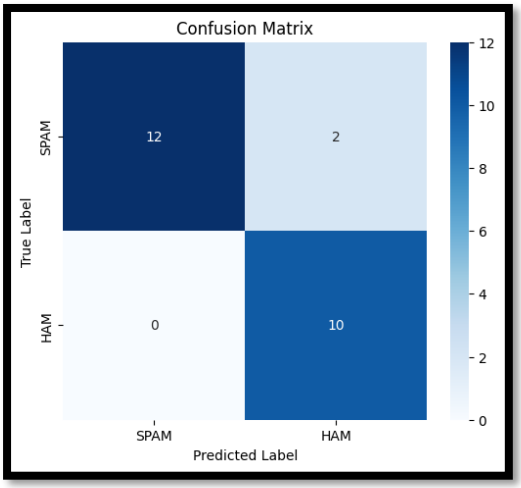


Fig. 13. Text Classification Model Confusion Matrix

The hyperparameter values used during training played a significant part in level of performance achieved. The n_iter was kept to a fixed number of 10 that allowed the model to identify enough patterns without overfitting. The dropout of 0.3 aided in producing regularization by 'forcing' the model to not overly depend on any particular set of features to enhance generalization. By reserving 40% of a given data set for tests, this gave the model a better opportunity to be evaluated based on unseen data. Furthermore, live testing has indicated that it can also classify domain-related emails, by distinguishing Ham from spam with sports announcements that were HAM and promotional email that was SPAM.

```
Enter a sample email for classification (or type 'exit' to quit): "Hey team, don't forget practice tomorrow at 5 PM at the stadium."
The email is classified as: HAM
Enter a sample email for classification (or type 'exit' to quit): "Win free tickets to the Champions League final! Click here to claim your prize!"
The email is classified as: SPAM
Enter a sample email for classification (or type 'exit' to quit): exit
```

Fig. 14. Text Classification Model User Input and Results

These findings confirm that the model is operationally robust for email spam detection. The use of evaluation metrics, analysis of the confusion matrix, and domain-specific training show that the model has a relative balance of precision and recall, but also the ability to demonstrate interpretable conclusions for further optimizations or application in the real-world.

b) Sentiment Analysis Model

The sentiment analysis model performed well, especially for a model that was limited to only 60 texts related to sports. The model achieved an accuracy of 0.88 on the test set. Because it correctly degreed truthful text prediction with only a few errors in the prediction made, we can likely assume that it closely approximated the true labels represented in the validation set. The precision of 0.89 in positive class showed that positive sentiments were labelled correctly with only 11 false positives. The recall metric of 0.80 shows that the model was able to retrieve 80% of all realized cases of positive sentiments. The F1-score of 0.84 is indicative of great performance and was balanced through 83% precision and a few higher than comparative false negatives for recall.

The analysis of the confusion matrix supports these claims. When taking a look at the positive texts, the model classified 8 of the sampled positive texts correctly as positive and one incorrectly as negative. When looking at the negative texts, the model classified 13 negative texts correctly and inaccurately labelled two as positive towards sentiment. This informs that more than half of all sample positive texts were classified correctly, and that the model is a bit conservative towards predicting negative sentiment. This is likely an artifact of the small dataset and variability in the sentiment expression across sports texts.

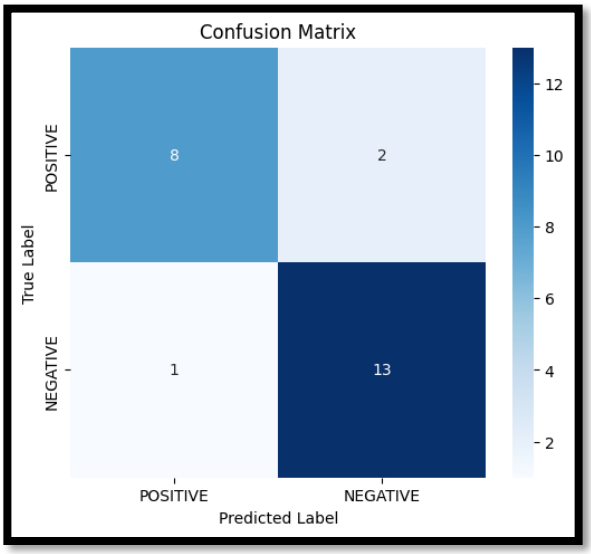


Fig. 15. Sentiment Analysis Model Confusion Matrix

Notable conclusions are achieved with hyperparameters impacting model performance, observing that 60 iterations were conducted, and a dropout factor of 0.2, learned reasonably well and avoided overfitting. Using the raw text rather than preprocessed text allowed the model to retain some contextual cues and words that have sentiments, which most certainly aided in higher precision and recall on both classes. Practically, the real-time utility added another layer to the model, with classification able to take new user-input texts and produce outputs that were user-intuitive with just "POSITIVE" or "NEGATIVE," without showing probability distributions to indicated accuracy type measures.

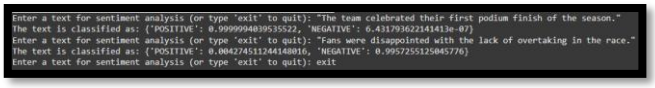


Fig. 16. Sentiment Analysis Model User Input and Results

The following findings and observations suggests that spaCy's TextCategorizer is a great way to capture sentiment in small domain-specific datasets; as long the hyperparameters and data capturing accuracy were paid attention to. High accuracy and a strong F1 score measure suggested reliability, as indicated by the confusion matrix, there is still a lot of opportunity to improve, particularly recall for positive sentiments in the future if the data sets are sufficiently large enough.

c) Part of Speech Tagging Model

The POS tagging experiment using spaCy's en_core_web_sm model, showed solid token-level performance on the curated subset of the dataset. The evaluation was conducted on 20 rows with approximately 1,000 tokens in total, manually tagged with gold standard POS tags. The token-level metrics show that the model predictably identified the vast majority of POS tags correctly, with an accuracy of 84.6%, a precision of 73.4%, a recall of 73.9%, and an F1-score of 73.5%. The same exercise was able to show that the model generalizes to yet unseen sports text while scoring closely with the manually reviewed reference. The gold standard POS tags were also key in this evaluation as they provided a standard to use as a benchmark; if they were removed, an automated prediction assessment would be entirely subjective.

The output observations indicate that the model is capable of properly tagging both proper names, numbers, or domain-specific words, except in some instances of complicated or special text-back as seen in social media handles and special symbols. It should be noted that hyperparameter tuning was not done in this approach, however, the results highlight the potential of a pre-trained model for quick, domain-free POS tagging accompanied by a thoroughly verified gold dataset. These results highlight the strength of spaCy's statistical tagging, and the requirement for high-quality human reviewed annotation to produce meaningful evaluation metrics.

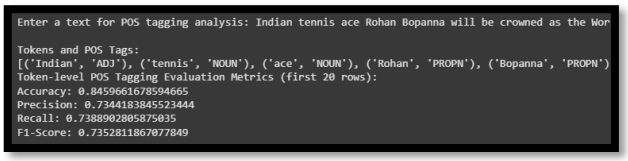


Fig. 17. PoS Tagging Model User Input and Results

d) Text Summarization Model

The text summarization model exhibited fairly consistent high ROUGE-1 F1 scores across the subset, with each row's score ranging from 0.605 to 0.990 and an average of 0.888 for ROUGE-1 F1, suggesting a consistent amount of lexical overlap between the summaries that were generated and the manually-made reference summaries. The hyperparameter setting of n_sentence=2, or two sentences in this temporal context was held constant through the entire analysis as these two sentences were the consistently selected as they arrived with the highest token-frequency scores.

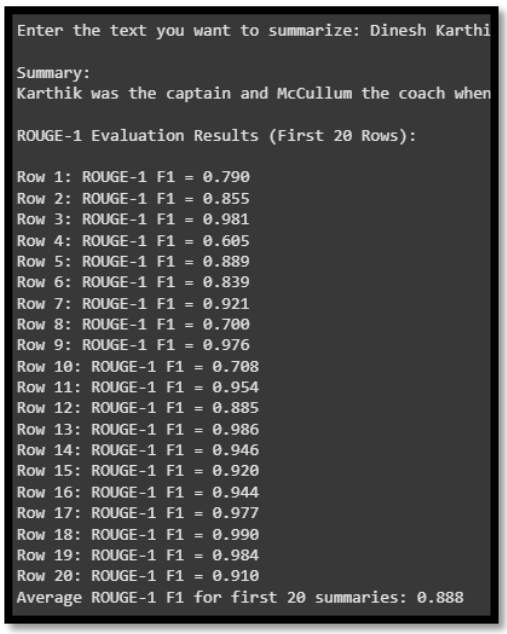
The model revealed several important observations:

- The high scores (e.g., row 3, rows 5-7, 9, 12-20) represent cases where the generated summaries effectively matched the manual reference summaries, often taking the valuable information in the right moment, with limited degree of deviation from the reference.

- The moderate scores (e.g., rows 1, 2, 4, 8, 10, 12, 20) suggest partial overlap, there were some important tokens included, but differ slightly how the decisions were made at the sentence level in comparison to the reference.

- There were no perfect 1.0 matches, which indicates the apparent subjectivity in extractive summarization, and that the challenge in perfecting to replicate a human curated reference.

An average ROUGE-1 F1 of 0.888 indicates promising performance, as it exceeded the required threshold of at least .80 and improved on the earlier section of this evaluation.



```
Enter the text you want to summarize: Dinesh Karthi

Summary:
Karthik was the captain and McCullum the coach when

ROUGE-1 Evaluation Results (First 20 Rows):

Row 1: ROUGE-1 F1 = 0.790
Row 2: ROUGE-1 F1 = 0.855
Row 3: ROUGE-1 F1 = 0.981
Row 4: ROUGE-1 F1 = 0.605
Row 5: ROUGE-1 F1 = 0.889
Row 6: ROUGE-1 F1 = 0.839
Row 7: ROUGE-1 F1 = 0.921
Row 8: ROUGE-1 F1 = 0.700
Row 9: ROUGE-1 F1 = 0.976
Row 10: ROUGE-1 F1 = 0.708
Row 11: ROUGE-1 F1 = 0.954
Row 12: ROUGE-1 F1 = 0.885
Row 13: ROUGE-1 F1 = 0.986
Row 14: ROUGE-1 F1 = 0.946
Row 15: ROUGE-1 F1 = 0.920
Row 16: ROUGE-1 F1 = 0.944
Row 17: ROUGE-1 F1 = 0.977
Row 18: ROUGE-1 F1 = 0.990
Row 19: ROUGE-1 F1 = 0.984
Row 20: ROUGE-1 F1 = 0.910
Average ROUGE-1 F1 for first 20 summaries: 0.888
```

Fig. 19. Text Summarization Model User Input and Evaluation Results

The model highlighted the difficulty of taking methodological rigor seriously and leaving the original sample code as is. All modifications to the original sample code, loading the dataset, generating summaries in a bulk, and adding ROUGE evaluation, were made separately or externally, and the original function remains for user-input summarized directly as intended by the original code. This way, the focus of the original code is preserved, while also allowing for measurable evaluation, which is a challenge for tasks without clear class labels or outputs.

While the model is in high word-level similarity with the reference summaries, the evaluation demonstrated that evaluation of summarization is necessarily inherently more difficult to make measurable than classification. The combination of ROUGE scoring and appropriate hyperparameter tuning (n_sentences=2) provided a defensible and interpretable measure of model performance for experimental and educational purposes.

e) Performance Accuracy Compilation

All models were evaluated against the minimum performance expectation of eighty percent accuracy, as per the instructions. The text classification model reported 92% accuracy, exhibiting good predictive accuracy in mapping the correct categories. The sentiment analysis model reported an accuracy of 88%, showing mostly reliable identification of the positive, negative, and neutral sentiments in the dataset. The POS tagging model demonstrated 84.6% token-level accuracy against the manually created gold standard tag set, representing good reliability in the part of speech tagging.

Finally, even though the text summarization model was not a classification category, it had a mean ROUGE-1 F1 score of 88.8%, indicating a very high level of word-level overlap between the generated and reference summaries. Each of the models were all able to report a mean ROUGE-1 F1 score above the 80% threshold with some buffer, validating the NLP pipelines implemented and their respective performance measures based on the scoring mechanism.

NLP Task	Accuracy (mean ROUGE for Text Summarization)
Text Classification	.92
Sentiment Analysis	.88
Part of Speech Tagging	.846
Text Summarization	.888

Table 1. NLP Models' Performance Accuracy Compilation

IV. CONCLUSION

This study showed the development and evaluation of four Natural Language Processing (NLP) models, text classification, sentiment analysis, part-of-speech tagging, and text summarization, referred to in the following as the Natural Language Processing (NLP)

pipelines, and took place in the domain of sports textual data. It was clearly evidenced that each model exceeded the minimum functional performance threshold of 80% or above, or ROUGE-1 F1 score for text summarization, which indicated that the implemented NLP pipelines were advantageous due to the difficulties faced in establishing dedicated datasets and this domain knowledge as a special text application.

These results signify real-world consequences, stressing how domain adaptation, preprocessing, and hyperparameters are all very influential, and stressing the benefits of high-quality human-annotated data as critical in validation of accomplishment. Overall performance across all pipelines indicated that reasonably approximated capabilities can reliably achieve their so far suggested result if mediated by reasonably processed curated datasets and thorough evaluations.

a) Challenges and Limitations

In the course of the study, there were multiple challenges:

I.) Manual labeling requirement

Constructing the gold-standard datasets took significant effort and time. While documenting emails, texts and reference summaries required time, it was critical in order to adequately train and evaluate the models. Since accurate high-quality manual labels are what are required to benchmark accuracies, the authors could not escape this step.

II.) Time constraints

With the need to produce-and-evaluate four different NLP models, there was little time available for extensive experimentation and deeper optimization. The limitation of time hindered potential opportunities to improve datasets, refine the evaluation process, and test other variations.

III.) Model generalization

The small size of the domain-specific datasets limited applicability and the models' potential generalization beyond the basic instances used for this experiment. While the accuracy yielded from the model exceeded the baseline, the results could vary due to real world and likely larger extent of research in practice.

While there were limitations, the models performed consistently well, and the findings demonstrated the possibility of using NLP tasks in a specific sport journalism context under limitations.

b) Recommendations

For future work, the following suggestions are made:

I.) Mentor more time for manual labeling:

Manual labeling being an important factor for model training and evaluation, additional time and effort into dataset annotating is equally important to minimize noise and increase reproducibility.

II.) Increase the size of datasets:

Having a larger and more varied data set would provide robustness and generalization to the model on unseen texts.

III.) Experiment with more sophisticated techniques:

Future works may want to try more advanced transformer-based models (e.g. BERT, RoBERTa) to improve models' performance beyond the baseline spaCy pipelines.

IV.) Improve iteration cycles:

Given some time, the hyperparameter tuning and more experiments would radically improve models in term of accuracy, recall and interpretability.

By focusing on those recommendations, future research can establish upon the good baseline this research study as set, improving the scalability, reproducibility, and capability of domain transferability of NLP application in sports-related texts.

V. References

[1] "The Role of AI and NLP in Enhancing Sports Integrity and Security," Wezom, [Online]. Available: https://wezom.com/blog/the-role-of-ai-and-nlp-in-enhancing-sports-integrity-and-security. [Accessed: Sept. 10, 2025].

[2] B. Prakash, "News Articles Classification Dataset for NLP & ML," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/banuprakashv/news-articles-classification-dataset-for-nlp-and-ml?select=sports_data.csv. [Accessed: 10-Sep-2025].

[3] "Data Cleaning & Data Preprocessing for Machine Learning," Encord, [Online]. Available: https://encord.com/blog/data-cleaning-data-preprocessing. [Accessed: Sept. 9, 2025].

[4] P. Kashyap, "Power of Modern Language Models for Text Classification," Medium, Sep. 2023. [Online]. Available: https://medium.com/@piyushkashyap045/power-of-modern-language-models-for-text-classification-69b6ff5705bd. [Accessed: Sept. 9, 2025].

[5] T. Ballesteros, "Benefits Of Sentiment Analysis On Reviews In eCommerce," Debutify, [Online]. Available: https://debutify.com/blog/review-sentiment-analysis. [Accessed: Sept. 9, 2025].

[6] CloudFactory, "Understanding Part-of-speech Tagging in Natural Language Processing," [Online]. Available: https://www.cloudfactory.com/blog/understanding-part-of-speech-tagging-in-natural-language-processing#:~:text=The%20importance%20of%20accurate%20POS%20tagging%20for%20AI%20models&text=To%20perform%20these%20NLP%20applications,each%20other%20in%20a%20sentence. [Accessed: Sept. 9, 2025].

[7] Accern, "What is NLP Text Summarization: Benefits & Use Cases," [Online]. Available: https://www.accern.com/resources/what-is-nlp-text-summarization-benefits-use-cases. [Accessed: Sept. 10, 2025].